

Industrial / Manufacturing Track -- Prompt Library

All prompts are copy-paste ready. Each has a primary version and a fallback.

Step 1: Database Setup

1.1 Primary Prompt

Please execute the setup script from the GitHub repository we connected. Run this file to create the industrial operations database, warehouse, all tables, internal stages, and load all sample data:

```
@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/industrial/scripts/setup.sql
```

Use EXECUTE IMMEDIATE FROM to run it. After execution, set INDUSTRIAL_HOL_WH as the active warehouse for the rest of our session and show me a summary table of every table created with its schema and row count so I can confirm everything loaded correctly.

1.1 Fallback Prompt

Please read the contents of the file at this Git stage path and execute it as a SQL script, running each statement sequentially:

```
@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/industrial/scripts/setup.sql
```

Break it into logical sections and show me progress after each section.

1.2 Inspect Prompt

Show me all the tables in INDUSTRIAL_HOL_DB.OPERATIONS and INDUSTRIAL_HOL_DB.ENERGY with their row counts. Display as a clean summary table.

Step 2: Semantic Models

2.1 Primary Prompt

Create a Cortex Analyst semantic view for the operations data in INDUSTRIAL_HOL_DB.OPERATIONS.

Include these tables: FACILITIES, PRODUCTION_LINES, TECHNICIANS,

EQUIPMENT, WORK_ORDERS, DOWNTIME_EVENTS, SPARE_PARTS, MAINTENANCE_LOGS, and QUALITY_INSPECTIONS.

Business intelligence rules:

- Flag "overdue maintenance" when NEXT_SERVICE_DATE < CURRENT_DATE
- Flag "critical equipment down" when STATUS = 'Overdue' and CRITICALITY = 'Critical'
- Flag "quality failure" when PASS_FAIL = 'Fail'

Make sure it can answer questions like:

1. "Which equipment is overdue for preventive maintenance?"
2. "What is the mean time to repair by facility?"
3. "Show me all unplanned downtime events this quarter"
4. "Which production lines have the highest defect rates?"

Name it EQUIPMENT_ANALYTICS and store it in INDUSTRIAL_HOL_DB.OPERATIONS.

Register it directly as a semantic view without saving any intermediate files to a stage.

2.1 Fallback Prompt

Copy the pre-built equipment analytics semantic model from GitHub:

Source:

@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/industrial/scripts/semantic_models/EQUIPMENT_ANALYTICS_MODEL.yaml

Destination: @INDUSTRIAL_HOL_DB.OPERATIONS.SEMANTIC_MODELS_STAGE

Use COPY FILES INTO to transfer it.

2.2 Energy Model

Create a Cortex Analyst semantic view for energy consumption analytics.

Include these tables:

- INDUSTRIAL_HOL_DB.ENERGY.ENERGY_METERS
- INDUSTRIAL_HOL_DB.OPERATIONS.FACILITIES

Join ENERGY_METERS to FACILITIES on FACILITY_ID.

Business intelligence rules:

- Flag "over budget" when ACTUAL_KWH > BUDGETED_KWH * 1.2
- Calculate budget variance as (ACTUAL_KWH - BUDGETED_KWH) / BUDGETED_KWH
- Track peak demand from PEAK_DEMAND_KW

Make sure it can answer questions like:

1. "Which facilities are over their energy budget?"
2. "What is the total energy cost by facility this year?"
3. "Show me peak demand trends by energy type"
4. "Which meters have the highest budget variance?"

Name it ENERGY_CONSUMPTION and store it in INDUSTRIAL_HOL_DB.ENERGY.

Register it directly as a semantic view without saving any intermediate files to a stage.

2.3 Explore -- Snowsight UI

Step 3: Cortex Search

3.1 Bring PDF into Snowflake

Bring the following PDF from our GitHub repository into INDUSTRIAL_HOL_DB.OPERATIONS so we can make it searchable:

@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/industrial/pdfs/Maintenance Operations Guide.pdf

3.2 Primary Prompt

Create a Cortex Search service called MAINTENANCE_DOCS_INFO in INDUSTRIAL_HOL_DB.OPERATIONS that makes the maintenance operations PDF searchable with natural language.

To do this:

- Temporarily scale up INDUSTRIAL_HOL_WH for processing power
- Read and parse the PDF content
- Break it into overlapping chunks for better search accuracy
- Index everything so it can be queried in plain English
- Scale the warehouse back down to SMALL when done

3.3 Preview the Document

Generate a presigned URL so I can preview the file "Maintenance Operations Guide.pdf" in INDUSTRIAL_HOL_DB.OPERATIONS.PDF_STAGE -- use an expiry of 3600 seconds.

3.4 Test Search -- Snowsight UI

Guide attendees to AI & ML > Cortex Search to test MAINTENANCE_DOCS_INFO.

Step 4: Intelligence Agent

4.1 Primary Prompt

Create a Snowflake Intelligence Agent named INDUSTRIAL_AGENT in INDUSTRIAL_HOL_DB.OPERATIONS.

Give it three tools:

1. A data analysis tool named EQUIPMENT_DATA connected to the EQUIPMENT_ANALYTICS semantic model in INDUSTRIAL_HOL_DB.OPERATIONS. It answers questions about equipment, maintenance, work orders,

downtime, and quality. Use INDUSTRIAL_HOL_WH.

2. A data analysis tool named ENERGY_DATA connected to the ENERGY_CONSUMPTION semantic model in INDUSTRIAL_HOL_DB.OPERATIONS. It analyzes energy consumption, costs, and budget variances across facilities. Use INDUSTRIAL_HOL_WH.
3. A document search tool named MAINTENANCE_DOCS connected to the MAINTENANCE_DOCS_INFO search service in INDUSTRIAL_HOL_DB.OPERATIONS. It searches maintenance documentation for procedures and policies.

4.2/4.3 -- Snowsight UI

Test in AI & ML > Agents > INDUSTRIAL_AGENT Playground.

Step 5: Streamlit App

5.1 Chat Page

Build a single-page Streamlit app in Snowflake called INDUSTRIAL_ASSISTANT_APP in INDUSTRIAL_HOL_DB.OPERATIONS using INDUSTRIAL_HOL_WH.

Use st.tabs() to create a horizontal tab bar at the top of the page with three tabs: "Chat", "Dashboard", and "Optimization". All three tabs should be visible and switchable at the top -- no sidebar page navigation.

Start with the Chat tab:

- A sidebar with the title "Operations Intelligence" and a brief description of the app
- A chat interface where users can have a conversation with INDUSTRIAL_AGENT
- Show the agent's responses in the chat as they arrive
- When the agent returns data, display it as a table below the response
- When the agent returns SQL, show it in a collapsible section
- Keep the conversation history visible throughout the session

Apply a modern industrial SaaS design -- think Siemens MindSphere or Rockwell FactoryTalk, not a dark video game UI:

- Main content background: clean off-white (#F8FAFC) with white cards and subtle drop shadows -- the primary surface should feel light and open
- Sidebar: dark slate (#1E293B) with white text and a steel blue left border on the active page -- the sidebar is the only dark area
- Primary color: steel blue (#2563EB) for headings, buttons, links, and active states
- Typography: sentence case throughout -- no all-caps; use bold weight for section headers, regular weight for body text
- Status indicators: green (#16A34A) for operational, amber (#D97706) for maintenance due -- reserve red (#DC2626) strictly for overdue or failed
- KPI cards: white background, steel blue top border, large bold metric number in dark slate (#1E293B), small gray label below
- Data tables: white background, light gray row dividers (#E2E8F0), status pills (small colored badges) instead of full-row color fills
- Charts: white background with steel blue as the primary series color
- Chat input area: white background matching the main content -- it should feel like a natural extension of the page, not a floating element
- Overall feel: clean, professional, and trustworthy -- the kind of dashboard a plant director would present in a boardroom, not a control room operator

5.2 Dashboard Page

Fill in the Dashboard tab of INDUSTRIAL_ASSISTANT_APP with the following content.

This page should show live data from INDUSTRIAL_HOL_DB:

TOP ROW - four summary numbers:

- Equipment Uptime Rate (operational equipment / total equipment as percentage)
- Open Work Orders (from WORK_ORDERS where STATUS in 'Overdue', 'Scheduled')
- Mean Time to Repair (average ACTUAL_HOURS from completed corrective work orders)
- Monthly Energy Cost (most recent month total from ENERGY.ENERGY_METERS)

MIDDLE ROW - two charts side by side:

- A bar chart of open work orders by facility, color-coded by priority (Critical, High, Medium, Low)
- A bar chart comparing budgeted vs actual energy consumption by facility for the most recent month

BOTTOM ROW - a critical alerts table:

- All equipment where NEXT_SERVICE_DATE < CURRENT_DATE, showing: Equipment Name, Facility, Criticality, Days Overdue, Last Service Date, Replacement Cost
- Sort by criticality (Critical first) then days overdue
- Include a button to download the table as a CSV

Chart styling:

Use plotly for all charts. Use green (#16A34A), amber (#D97706), and red (#DC2626) to represent operational status (good/warning/critical). For non-status charts use steel blue (#2563EB) as the primary series color and slate gray (#64748B) as the secondary. All chart backgrounds should be white (FFFFFF) with light gray grid lines. No dark backgrounds on any chart.

Position the legend below each chart (orientation="h", y=-0.3) to prevent overlap with chart titles. Add a top margin (t=50) on all charts so the title never crowds the plot area.

5.3 Optimization Page

Fill in the Optimization tab of INDUSTRIAL_ASSISTANT_APP with the following content.

Title: "Operations Intelligence"

Subtitle: "Schedule maintenance and forecast energy consumption using AI and machine learning on your live operational data."

At the top of the tab, add a mode selector using radio buttons:

- "Maintenance Scheduler"
- "Energy Forecaster"

MODE 1: Maintenance Scheduler

When this mode is selected, show the following controls in a row above the results (not in a sidebar):

- A multi-select for Facility (options from FACILITIES table, plus "All")
- A segmented button for Time Horizon: 7 days / 14 days / 30 days
- A slider labeled "Priority" with left end "Minimize Downtime Risk" and right end "Minimize Labor Cost" (0-100 range)
- A steel blue "Run Optimizer" button

When the button is clicked:

1. Pull overdue and upcoming work orders for the selected facilities

- within the selected time horizon from WORK_ORDERS and EQUIPMENT
2. Pull technician availability and specialization from TECHNICIANS
 3. Use linear programming (scipy) to assign work orders to technicians, weighted by the priority slider (0 = weight by criticality and overdue days, 100 = weight by technician hourly rate)
 4. Respect constraints: same facility only, one job at a time, max 10 hours per technician per day

Display results as:

- Three KPI cards: Work Orders Scheduled, Days to Clear Backlog, Total Labor Hours
- A plotly Gantt chart (px.timeline) with:
 - Y-axis: technician names
 - X-axis: scheduled dates
 - Bars colored by equipment criticality using the status palette (green=#16A34A for Low, amber=#D97706 for Medium, orange=#F97316 for High, red=#DC2626 for Critical)
 - Hover tooltip showing: equipment name, work order type, estimated hours
- A before/after comparison: overdue count before vs after the schedule

MODE 2: Energy Forecaster

When this mode is selected, show the following controls:

- A dropdown to select Facility (from FACILITIES table)
- Radio buttons for Forecast Horizon: 1 month / 2 months / 3 months
- A steel blue "Generate Forecast" button

When the button is clicked:

1. Pull monthly energy data from ENERGY_METERS for the selected facility, converting the MONTH column (YYYY-MM format) to a proper DATE value
2. Use SNOWFLAKE.ML.FORECAST to train a forecast model on the historical ACTUAL_KWH values and predict the next N months (based on the selected horizon). Run this as a SQL operation inside Snowflake.
3. Retrieve the forecast results including the predicted value and the upper/lower confidence interval bounds

Display results as:

- A headline KPI card: "Forecasted Monthly Cost" (forecasted kWh multiplied by average historical COST_PER_KWH for the facility)
- A plotly line chart with three series:
 - Historical actual kWh (solid steel blue line, #2563EB)
 - Historical budget kWh (dashed gray line, #94A3B8)
 - Forecasted kWh (dotted line with a light blue shaded confidence band)
 - X-axis: months, Y-axis: kWh consumed
 - Legend positioned below the chart, white background
- A "Budget Risk" table showing months where the forecast exceeds the historical average budget by more than 10%, with columns: Month, Forecasted kWh, Budget kWh, Variance %, Risk Level
- A small note below: "Forecast based on [N] months of historical data. Wider confidence intervals reflect limited history."

Apply the same styling as the rest of the app throughout this tab: white background, steel blue primary, plotly charts with white backgrounds, sentence case labels, no all-caps.

Add scipy to the app's dependencies.